Info I, Klausur Wintersemester 23/24

1. Allgemeine Fragen

- Speicherbereiche nach Größe sortieren. Lösung: Register, Prozessorcache, Festplatte
- Typischen Ablauf eines Algorithmus beschreiben. Lösung: Initialisiere, Iteriere, Terminiere
- Java-Codeelemente nach Umfang/Größe sortieren. Lösung: Methode, Klasse, Bibliothek
- Prozess beim Schreiben von Programmen beschreiben. Lösung: Editiere, Compiliere, Teste
- `java Intervallsumme A B` soll die Summe der natürlichen Zahlen zwischen `A` und `B` ausgeben. (Es muss kein eigener Code geschrieben werden. Stattdessen sollen die Zeilen des Programms per Dropdown-Menü ausgewählt werden.)
  Lösung:

```java
class Intervallsumme {
    public static void main(String[] args) {
        int s = 0;
        int min = Integer.parseInt(args[0]); int max = Integer.parseInt(args[1]
        int n = min;
        while (max >= n) {
            s += n; n++; // alternativ: n += 1; s += n;
        }
        System.out.println(s);
    }
}
```

- Beim Compilieren eines Java-Programms entsteht _____. Lösung: Bytecode

2. Vermischtes zu Klassentypen

Kommentar der/des Prüfungsverantwortlichen:

Lösung

1. überladen
2. `int[] i = {1.500,000}` ist fehlerhaft, denn `1.500` ist ein Gleitpunktliteral.
3. 4 mal: `a` durchläuft der Reihe nach die Werte 10, 9, 8, 7. Dann erst hat `run` den Wert `false`, d.h. die Abbruchbedingung ist erreicht.
4. a.length ist 2, a[0].length ist 3. Deswegen verursacht int c = a[0][a.length]; KEINEN Fehler und boolean b hat in beiden Zeilen den Wert true. Die einizige korrekte Aussage ist: Die Zeile a = new int[6]; verursacht einen Fehler (einen Compilezeitfehler, weil hierbei an a eine 1D-Referenz zugewiesen werden soll, a aber als 2D-Referenz deklariert ist).
5. Jeder Konstruktoraufruf erzeugt ein neues Objekt mit eigener Adresse. Der Vergleich bezieht sich auf die Referenzen (Adressen) der Objekte, nicht ihren Wert (hier: allein der Wert des `honor`-Attributs). Also lautet die Ausgabe `false` (1P).
6. Richtige Antwort (2P): Die Klasse kann nicht compiliert werden, denn es gibt keinen Konstruktor mit passenden Argumenttypen.
7. Ergänzte Klasse

```
1: public class C {
2:     private int a;
3:     public C(int a) {
4:         this.a=a;     // initialisiere Attribut mit uebergebenem Parame
5:     }
6:     private void f() {
7:         a *= 2;
8:         System.out.println("a=" + a);
9:     }
10:    public static void main(String[] args) {
11:        C c=new C(5); // erzeuge ein Objekt der Klasse C mit Attributwert
12:                      // ... verwenden Sie dafuer den Bezeichner c
13:        c.f();        // Methode f() aufrufen
14:    }
15: }
```

8. Erzeugte Ausgabe `a=10`

Aufgaben zu 5. und 6. (Bild: Altklausurensammlung):

1. Objekte verstehen:

```
public class Ninja {
    private int honor;
    public Ninja( int s) {
        honor = s;
    }
}
```

Zwei `Ninja`-Objekte werden instanziiert:

```
Ninja a = new Ninja(20), b = new Ninja(20);
```

Welchen Wert hat nun der Ausdruck `a == b`?

2. Fehler oder ...?

```
public class SillyTestClass {
    private int x, y;
    public SillyTestClass(int x, int y) {
        System.out.println(y);
    }
    public SillyTestClass(String string1, String string2) {
        System.out.println(string1 + string2);
    }
    public static void main (String[] args) {
        SillyTestClass app = new SillyTestClass(2, "Tests");
    }
}
```

Welche Aussage charakterisiert diesen Code am besten?

3. Fehler oder ...?

Bestmögliche Lösung:

Erzeugt die untenstehende Klasse

- einen Compilezeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): -
- einen Laufzeitfehler? Falls ja, so tragen Sie die Nummer der fehlerhaften Zeile ein, ansonsten einen Bindestrich (-): 6
- eine Ausgabe? Falls ja, so tragen Sie die Ausgabe ein, ansonsten einen Bindestrich (-): -

Für volle Punktzahl müssen Sie an jeder Stelle eine Auswahl treffen!

```
1:  class FehlerOderWas {
2:      public static void main(String[] args) {
3:          int[] a = { 2, 3, 5, 7, 11};
4:          int[] b = new int[6];
5:          b = a;
6:          a[0] = b[5];
7:          System.out.println(5+a[0] % 2);
8:      }
9:  }
```

Kommentar der/des Prüfungsverantwortlichen:

Lösungsweg:

- ArrayIndexOutOfBoundsException: 5 (Zugriff jenseits der Arraygrenzen) in Zeile 6

Erklärung: Der Code ist syntaktisch korrekt, darum kein Compilezeitfehler. Durch Zeile 5 verweist b auf die gleiche Speicheradresse wie a, also auf ein Array der Länge 5. Der Zugriff b[5] dagegen erfordert mindestens ein Array der Länge 6 (0-basierte Zählung). Da dieser Laufzeitfehler vor der Ausgabeanweisung auftritt, wird diese nicht mehr ausgeführt.

4. Der Datentyp char (Bild: Aufgabensammlung)

char-Merkwürdigkeiten (I1-ID: 8w1aucc1tng0)

In folgendem Quelltext gibt es mindestens eine Zeile, die der Compiler nicht akzeptiert. Streichen Sie alle betroffenen Zeilen. Welche Textausgaben erzeugen dann die verbleibenden Zeilen?

```
1: char c = 'a';  System.out.println(c);
2: c = c+1; System.out.println(c);
3: System.out.println('c'+1);
4: System.out.println((char)('c'+1));
```

(Die falsche Zeile soll hier mit "falsch" markiert werden)
Lösung: 1. a, 2. falsch, 3. 100, 4. d

5. Zahlensysteme

- Dezimalzahl 2020 ins Binärsystem umwandeln. Lösung: 11111100100
- Hexadezimalzahl 2020 ins Dezimalsystem umwandeln. Lösung: 8224
- Oktalzahl 2020 ins Hexadezimalsystem umwandeln. Lösung: 410

6. Mystery-Rekursion, Schiefe H-Bäume

Teil A

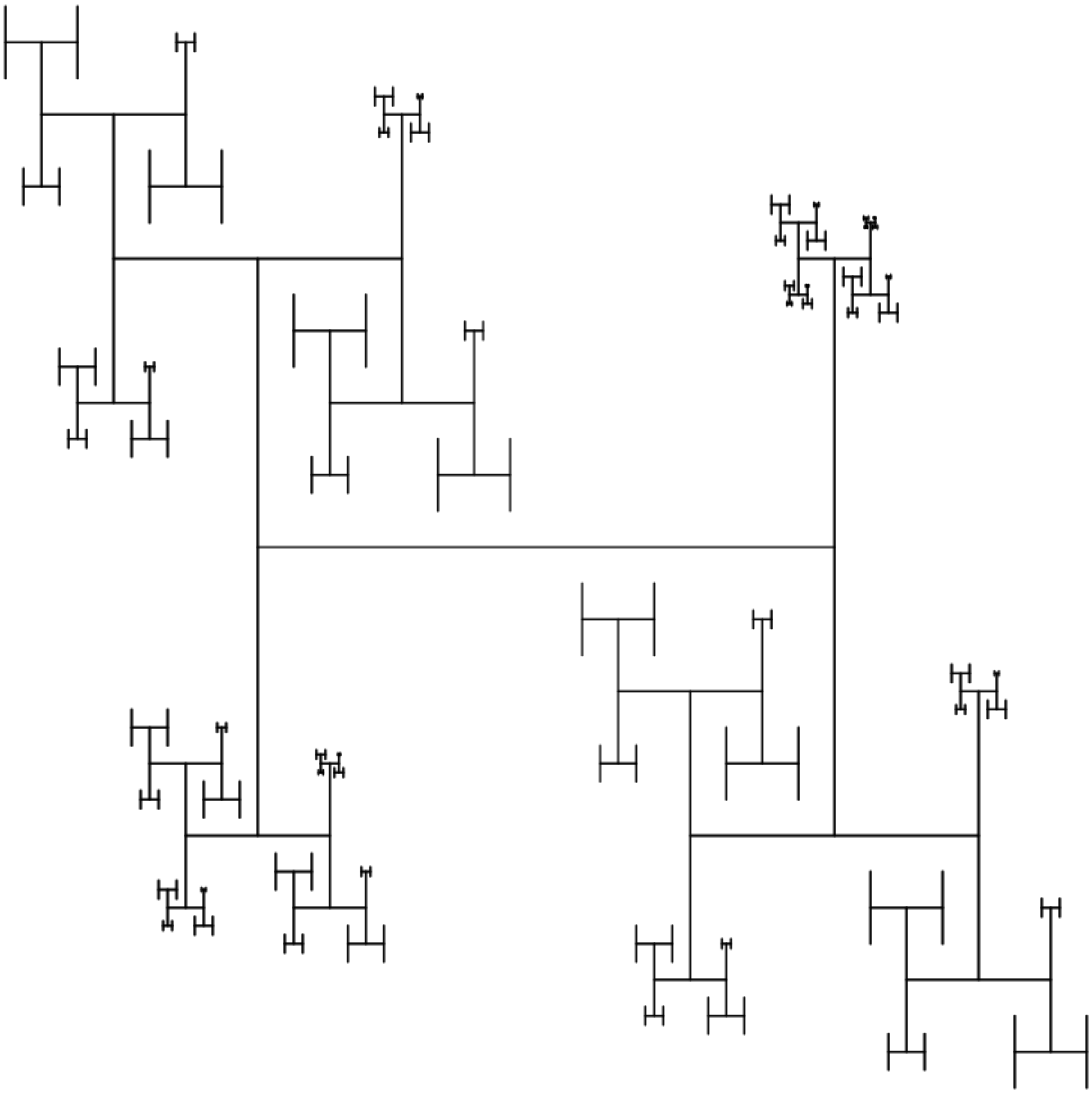
```
1: public static int mystery(int n){
2:     if(n == 0){
3:         return 0;
4:     }
5:
6:     return mystery(n-1) + 2*n-1;
7: }
```

1. Was bewirkt der Aufruf `mystery(5)`? Lösung: Ergebnis 25 wird geliefert
2. Was bewirkt der Aufruf `mystery(-5)`? Lösung: java.lang.StackOverflowError
3. Welcher Code muss in Zeile 5 eingefügt werden, damit `mystery(-n) == -mystery(n)` für alle n gilt? Lösung: `if(n < 0) return mystery(n+1) + 2*n+1;`

Teil B

```
1:  public static void draw(int n, double x, double y, double size) {
2:      if (n == 0) return;
3:      double x0 = x - size/2, x1 = x + size/2;
4:      double y0 = y - size/2, y1 = y + size/2;
5:      StdDraw.line( x0, y0, x0, y1);
6:      StdDraw.line( x1, y0, x1, y1);
7:      StdDraw.line( x0, y, x1, y);
8:
9:      // zeichne rekursiv 4 kleinere H-Baeume der Ordnung n-1
10:     draw(n-1, x0, y1, 0.5 * size);
11:     draw(n-1, x0, y0, a * size);
12:     draw(n-1, x1, y1, b * size);
13:     draw(n-1, x1, y0, 0.5 * size);
14: }
```

Was muss in Zeile 8 eingefügt werden, damit der Aufruf `draw(4, 0.5, 0.5, 0.5)` das folgende Bild eines schiefen H-Baumes ergibt?



Lösung: `double a = 0.25, b = 0.125;`

7. Lokale Variablen (Bild: Altklausurensammlung)

7. 3. Statische Methoden (l1-ID: 0f1dmv607ih0)

```
public class StatischeMethoden {
        public static void q1(int n) {
                n = n * n;
         }
        public static void q2(int[] f, int i) {
                f[i] = f[i] * f[i];
        }
        public static void main(String[] args) {
                int n = 2;
                 int[] f = { n, n, n};
                for (int i = 0; i < 3; i++) {
                        q1(n); q2(f,n);
                }
                System.out.println(n);
                System.out.println(f[n]);
         }
 }
```

Wie lautet die erste Ausgabezeile?

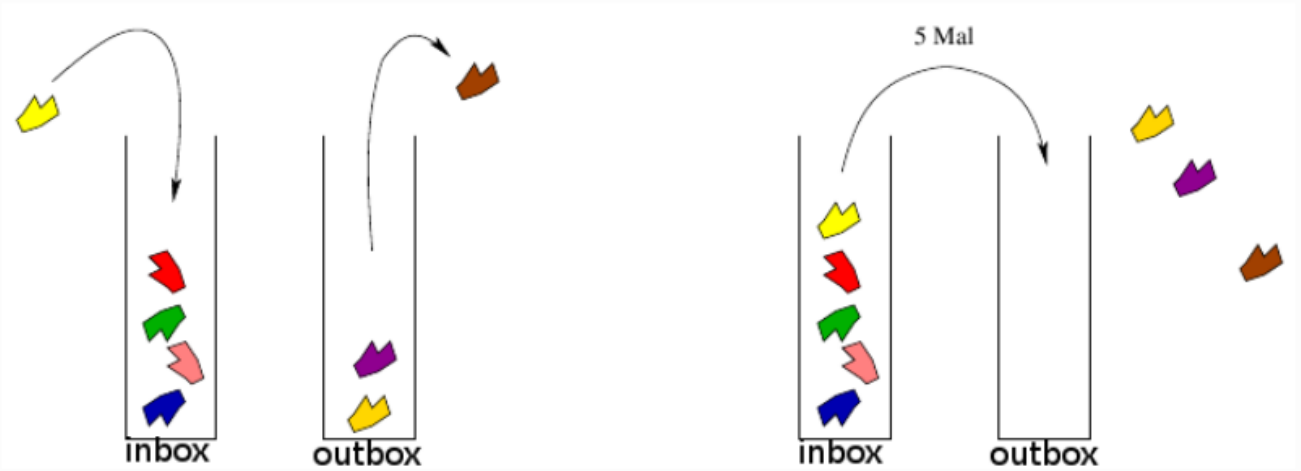
2

Wie lautet die zweite Ausgabezeile?

256

8. Queue aus zwei Stacks (Bild: Aufgabensammlung)

Queue aus zwei Stacks bauen (I1-ID: a4g40y31x4h0)



Mit zwei Stacks kann man eine Queue implementieren:

- `enqueue(...)` entspricht Einspeichern in `inbox`
- `dequeue()` entspricht Ausspeichern aus `outbox`.

Unter Verwendung dieser API

`public class Stack`	
`boolean empty()`	testet, ob dieser Stack leer ist
`void push(String s)`	speichert `s` in diesem Stack
`String pop()`	liefert zuletzt gespeicherten String aus diesem Stack und entfernt ihn

sähe die entsprechende Klasse z.B. so aus:

```
public class ZSQueue { // Zwei-Stack-Queue
    private Stack inbox, outbox;
    public ZSQueue() { ... }
    public boolean empty() { ... }        // Leerheitstest
    public void enqueue(String s) { ... } // einspeichern
    public String dequeue() {             // ausspeichern und liefern
        // Rumpf (Ihre Aufgabe)
    }
}
```

Programmieren Sie den Rumpf der Methode `dequeue()`:

- ist die Queue leer, ist Ausspeichern nicht möglich!
- teste ob `outbox` leer ist und fülle in diesem Fall den kompletten Inhalt von `inbox` um in `outbox`, dann ein Element aus `outbox` ausspeichern und liefern.

(Es müssen wieder die Zeilen des Programms per Dropdown-Menü ausgewählt werden; daher die fehlende Einrückung)
Lösung:

```
if(outbox.empty())
while(!inbox.empty())
outbox.push(inbox.pop());
if(!outbox.empty())
return outbox.pop();
throw new RuntimeException("Queue ist leer");
```

9. Objektorientierung

- Wodurch wird Kapselung bei der objektorientierten Programmierung erreicht?
  Durch _____ Instanzvariablen und _____ Methoden.
  Lösung: 1. private, 2. public
- Welche der Konzepte (A) Vererbung, (B) Kapselung, (C) Überladen kann man anhand folgender Situationsbeschreibung illustrieren?
  Jedes Tier kann einen Laut von sich geben (ein Geräusch machen). Jeder Hund ist ein Tier und kann einen Laut von sich geben, nämlich: Wau Wau! Jede Katze ist ein Tier und kann einen Laut von sich geben, nämlich: Miau!
  Lösung: (A) ist richtig, (B) und (C) sind falsch

10. Sortieralgorithmen

Ihre Antwort:

Betrachten Sie folgendes Code für einen Sortieralgorithmus, der ein Array f der Länge N sortiert.

```
public static void sort(int[] f){
    int min, tmp;
    for(int i = 0; i < f.length; i++){
        min = i;
        for(int j = i; j < f.length; j++){
            if(f[min] > f[j])
                min = j;
        }
        tmp = f[i];
        f[i] = f[min];
        f[min] = tmp;
    }
}
```

Die algorithmische Idee sollte Ihnen bekannt vorkommen. Was können Sie darüber aussagen?

Dieser Algorithmus ist Vergleichsbasiert ✓ .

Wieviel Speicherplatz benötigt der Algorithmus (zusätzlich zum Array f)? Das hängt nicht von N ab. ✓ .

Die Laufzeit des Algorithmus im besten Fall ist O(N^2) ✓ .

Die Laufzeit des Algorithmus im schlechtesten Fall ist O(N^2) ✓ .

Kommentar der/des Prüfungsverantwortlichen:

Es ist ein vergleichsbasiertes Verfahren: die Bedingung in der if-Anweisung ist ein Vergleich zweier Array-Werte.

Zusätzlich zu f werden die Variablen min, tmp, i, j benutzt: konstant viel zusätzlicher Speicher.

Dem Code ist anzusehen, dass die äußere Schleife N mal iteriert, die innere 1, 2, ..., N mal - je nach Wert von i. Die Gaussche Summe führt zur Schrittzahl O(N^2).

[Das ist eine bekannte Eigenschaft von Selection-Sort.]

11. Ist ein bestimmtes Wort im Suchsel-Rätsel enthalten?

Ist ein bestimmtes Wort im Suchsel-Rätsel enthalten? (I1-ID: 7ub80vo093k0) - Full score: 18

Description

"Suchsels" bestehen aus einer \(N\times N\)-Anordnung von Buchstaben und einer Liste von Suchwörtern, die darin von links nach rechts oder von oben nach unten zu lesen sind. Hier ein Beispiel (mit Zeilen- und Spaltennummern):

```
Lösung                          Rätsel

   0 1 2 3                         0 1 2 3
0: F                            0: F I L Z
1: E N T E                      1: E N T E
2: I                            2: I F E K
3: N O T E                      3: N O T E
```

Die Lösung besteht aus einem zweidimensionalen `char`-Array, das die Buchstaben der Suchwörter an den richtigen Stellen enthält, ansonsten nur Leerzeichen. Das Rätsel besteht ebenfalls aus einem zweidimensionalen `char`-Array, nur sind die Leerzeichen zufällig durch Buchstaben ersetzt. Im Beispiel sind dadurch weitere sinnvolle Wörter entstanden: `FILZ` (ab Position (0,0) von links nach rechts) und `INFO` (ab (0,1) von oben nach unten).

Aufgabe

Programmieren Sie zwei statische Methoden

```
public static boolean containsLR(String w, int x, int y, char[][] r)
public static boolean containsLR(String w, char[][] r)
```

- Hierbei speichert `r` die Buchstaben des Rätsels und `w` ist das zu prüfende Wort.
- Die erste Methode prüft, ob `w` *ab Position* `(x,y)` von links nach rechts in `r` zu finden ist (im Beispiel würde damit FILZ gefunden werden, nicht aber INFO).
- Die zweite Methode ruft die erste Methode mehrmals auf, um zu prüfen, ob `w` *irgendwo* in `r` von links nach rechts zu finden ist.

Hinweise

1. Behandeln Sie bei der ersten Methode am besten zuerst den Fall, dass `w` zu lang ist, um ab Position `(x,y)` in `r` zu passen (welcher Wert muss dann geliefert werden?).
2. Auf die Zeichen von `w` können Sie mit `w.charAt(int pos)` zugreifen (siehe Skript Seite 168f.).
3. Programmieren Sie die beiden Methoden in untenstehendem "Klausureditor". Bitte nicht in eine Klasse einbetten.

Score: 18

```java
public static boolean containsLR(String w, int x, int y, char[][] r){
    if(y + w.length() > r[x].length){
        return false;
    }

    for(int i = 0; i < w.length(); i++){
        if(w.charAt(i) != r[x][y + i]){
            return false;
        }
    }

    return true;
}

public static boolean containsLR(String w, char[][] r){
    for(int i = 0; i < r.length; i++){
        for(int j = 0; j < r[i].length; j++){
            if(containsLR(w, i, j, r)){
                return true;
            }
        }
    }

    return false;
}
```

12. Klasse Gerade

Klasse Gerade (I1-ID: 0hl56v31lvh2) - Full score: 12

Description

Eine **Gerade** \(y = m\cdot x + n\) in der Ebene soll allein durch `m` und `n` gegeben sein. Schreiben Sie eine Java-Klasse für *unveränderliche* `Gerade`-Objekte, die folgende API realisiert:

```
| public class Gerade                |
|------------------------------------+-------------------------------------------------------------
| public Gerade(double m, double n)| erzeugt neues Gerade-Objekt
| public static int anzahl()       | liefert die Anzahl erzeugter Gerade-Objekte
| public double getM()             | liefert m dieser Gerade
| public double getN()             | liefert n dieser Gerade
| public String toString()         | liefert String-Darstellung wie im Beispiel unten
```

Beispiel: Der String zur Geraden \(y = 2\cdot x + 3\) lautet `y = 2*x + 3`.

Score: 12

```java
public class Gerade {
    private final double m;
    private final double n;
    private static int anzahl = 0;

    public Gerade(double m, double n){
        this.m = m;
        this.n = n;

        anzahl++;
    }

    public static int anzahl(){
        return anzahl;
    }

    public double getM(){
        return this.m;
    }

    public double getN(){
        return this.n;
    }

    public String toString(){
        return "y = " + this.m + "*x + " + this.n;
    }
}
```